

Department of Computer Science & Engineering**Microprocessor & Computer Architecture****UE24CS251B****MPCA Lab Programs - 6**

Name: DHRITI KAMANI		SRN: PES1UG24AM085	Section: B
1	Write an ARM7TDMI assembly program using nested procedure calls to compute the sum of squares of two numbers. The main program calls a procedure SUM_SQ, which in turn calls another procedure SQUARE to compute the square of a number. Program and Results Screenshot: <pre>.global _start _start: MOV R1, #3 MOV R2, #4 BL SUM_SQ STOP: B STOP SUM_SQ: STMFD SP!, {R4, R5, LR} MOV R0, R1 BL SQUARE MOV R4, R0 MOV R0, R2 BL SQUARE MOV R5, R0 ADD R0, R4, R5 LDMFD SP!, {R4, R5, PC} SQUARE: STMFD SP!, {LR} MUL R0, R0, R0 LDMFD SP!, {PC}</pre>		

Registers

Refresh

r0	00000019
r1	00000003
r2	00000004
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000028
pc	0000000c
cpsr	000001d3 NZCVI SVC
spsr	00000000 NZCVI ?

Registers

Call stack

Trace

Breakpoints

Watchpoints

Symbols

Counters

Settings

Number Display Options

Size: Word

Format: Hexadecimal

Memory words per row: 4

Disassembly (Ctrl-D)

Go to address, label, or register: 00000000 Refresh

Address	Opcode	Disassembly
ffffffdc	aaaaaaaa	bge 0xfeaaaa8c
ffffffe0	aaaaaaaa	bge 0xfeaaaa90
ffffffe4	aaaaaaaa	bge 0xfeaaaa94
ffffffe8	aaaaaaaa	bge 0xfeaaaa98
ffffffec	aaaaaaaa	bge 0xfeaaaa9c
fffffff0	aaaaaaaa	bge 0xfeaaaaa0
fffffff4	aaaaaaaa	bge 0xfeaaaaa4
fffffff8	aaaaaaaa	bge 0xfeaaaaa8
fffffffc	aaaaaaaa	bge 0xfeaaaaac
2 .global _start		
4 _start:		
5 _start:		
00000000	e3a01003	5 mov r1, #3 ; 0x3
00000004	e3a02004	6 mov r2, #4 ; 0x4
7 BL SUM_SQ		
00000008	eb000000	bl 0x10 (0x10: SUM_SQ)
9 STOP:		
10 B		STOP
STOP:		
0000000c	ea000000	b 0xc (0xc: STOP)
13 SUM_SQ:		
00000010	e92d4030	14 STMFD SP!, {R4, R5, LR}
00000014	e1a00001	SUM_SQ:
		push {r4, r5, lr}
		16 mov r0, r1
		17 BL SQUARE
		bl 0x14 (0x14: SQUARE)

Editor (Ctrl-E)

Disassembly (Ctrl-D)

Memory (Ctrl-M)

Messages

CP-Editor has started with system **ARMv7 generic** containing a **ARMv7** processor.

Compiling...

Code and data loaded from ELF executable into memory. Total size is 64 bytes.

Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfpu=neon-fp16 -gdwarf2 -o work/asmwQ56un.s.o work/asmwQ56un.s

Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmwQ56un.s.elf work/asmwQ56un.s.o

Compile succeeded.

- 2 **Write an ARM7TDMI assembly language program to multiply two matrices of suitable order and store the result in a separate memory location.**

Program and Results Screenshot:

```

.global _start
_start:
    LDR    R0, =MAT_A
    LDR    R1, =MAT_B
    LDR    R2, =MAT_C
    MOV    R3, #0
ROW_LOOP:
    CMP    R3, #2
    BEQ    STOP
    MOV    R4, #0
COL_LOOP:
    CMP    R4, #2
    BEQ    NEXT_ROW
    MOV    R5, #0
    MOV    R8, #0

```

K_LOOP:

```
CMP    R5, #2
BEQ    STORE
MLA     R6, R3, R9, R5
LSL     R6, R6, #2
LDR     R10, [R0, R6]
MLA     R7, R5, R9, R4
LSL     R7, R7, #2
LDR     R11, [R1, R7]
MUL     R12, R10, R11
ADD     R8, R8, R12
ADD     R5, R5, #1
B       K_LOOP
```

STORE:

```
MLA     R6, R3, R9, R4
LSL     R6, R6, #2
STR     R8, [R2, R6]
ADD     R4, R4, #1
B       COL_LOOP
```

NEXT_ROW:

```
ADD     R3, R3, #1
B       ROW_LOOP
```

STOP:

```
B       STOP
MOV     R9, #2
```

MAT_A:

```
.word  1, 2
.word  3, 4
```

MAT_B:

```
.word  5, 6
.word  7, 8
```

MAT_C:

```
.space 16
```

Registers

Refresh

r0	00000080
r1	00000090
r2	000000a0
r3	00000002
r4	00000002
r5	00000002
r6	00000004
r7	00000004
r8	00000012
r9	00000000
r10	00000002
r11	00000006
r12	0000000c
sp	00000000
lr	00000000
pc	00000078
cpsr	600001d3 NZCVI SVC
spsr	00000000 NZCVI ?

Registers

Call stack

Trace

Breakpoints

Watchpoints

Symbols

Counters

Settings

Number Display Options

Size: Word

Format: Hexadecimal

Memory words per row: 4

Disassembly (Ctrl-D)

Go to address, label, or register: 00000078 Refresh

Address	Opcode	Disassembly
00000064	e7828006	45 str r8, [r2, r6]
00000068	e2844001	47 add r4, r4, #1 ; 0x1
		48 B COL_LOOP
0000006c	eaffffea	b 0x1c (0x1c: COL_LOOP)
		50 NEXT_ROW:
00000070	e2833001	51 add r3, r3, #1 ; 0x1
		52 B ROW_LOOP
00000074	eafffffe	b 0x10 (0x10: ROW_LOOP)
		54 STOP:
		55 B STOP
00000078	eafffffe	b 0x78 (0x78: STOP)
0000007c	e3a09002	58 mov r9, #2 ; 0x2
		MAT_A:
00000080	00000001	andeq r0, r0, r1
00000084	00000002	andeq r0, r0, r2
00000088	00000003	andeq r0, r0, r3
0000008c	00000004	andeq r0, r0, r4
		MAT_B:
00000090	00000005	andeq r0, r0, r5
00000094	00000006	andeq r0, r0, r6
00000098	00000007	andeq r0, r0, r7
0000009c	00000008	andeq r0, r0, r8
		MAT_C:
000000a0	0000000f	andeq r0, r0, pc
000000a4	00000012	andeq r0, r0, r2

Editor (Ctrl-E)

Disassembly (Ctrl-D)

Memory (Ctrl-M)

Messages

CR-Unit has started with system ARMv7-generic containing a ARMv7 processor.

Compiling...

Code and data loaded from ELF executable into memory. Total size is 192 bytes.

Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfpu=neon-fp16 --gdwarf2 -o work/asmGE1h89.s.o work/asmGE1h89.s

Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmGE1h89.s.elf work/asmGE1h89.s.o

Compile succeeded.

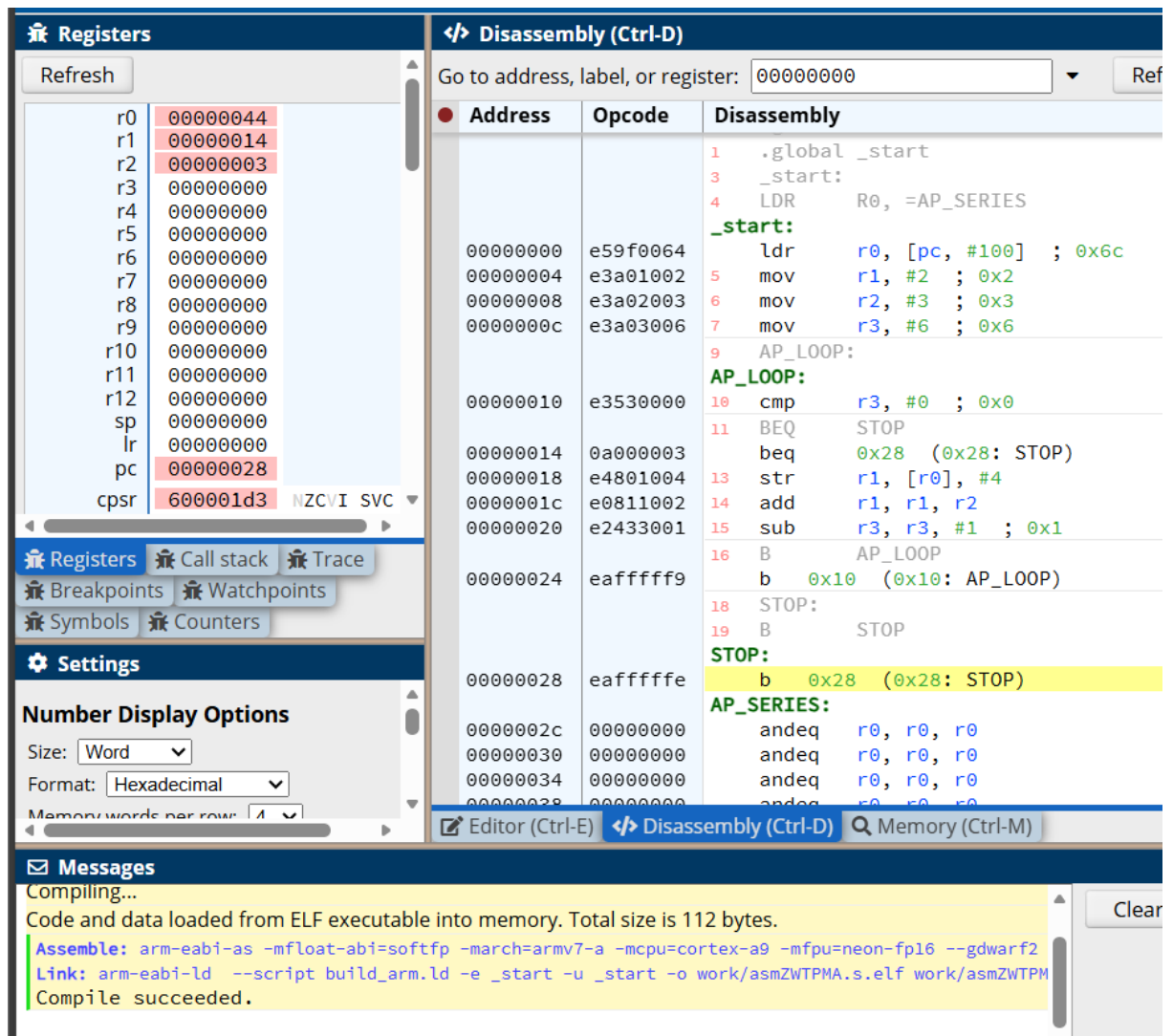
3 Write an ARM7TDMI assembly language program to generate the first N terms of an arithmetic progression and store them in memory.

Program and Results Screenshot:

```

.global _start
_start:
    LDR    R0, =AP_SERIES
    MOV    R1, #2
    MOV    R2, #3
    MOV    R3, #6
AP_LOOP:
    CMP    R3, #0
    BEQ    STOP
    STR    R1, [R0], #4
    ADD    R1, R1, R2
    SUB    R3, R3, #1
    B      AP_LOOP
STOP:
    B      STOP
  
```

AP_SERIES:
 .space 64



The screenshot shows an ARM development tool interface with the following components:

- Registers Panel:** Lists registers r0 through r12, sp, lr, pc, and cpsr. The pc register is highlighted with the value 00000028. The cpsr register is highlighted with the value 600001d3.
- Disassembly Panel:** Shows assembly code starting from address 00000000. The code includes labels like _start, AP_LOOP, and STOP. The instruction at address 00000028 is highlighted: `b 0x28 (0x28: STOP)`.
- Messages Panel:** Displays compilation messages, including "Code and data loaded from ELF executable into memory. Total size is 112 bytes." and "Compile succeeded."

- 4 **Write an ARM7TDMI assembly language program to compute the sum of digits of a given number using repeated division.**

Program and Results Screenshot:

```

.global _start
_start:
    MOV    R0, #1234
    MOV    R5, #0
MAIN_LOOP:
    CMP    R0, #0
    BEQ    STOP
    MOV    R1, #0
  
```

```

MOV    R2, R0
MOV    R3, #10
DIV_LOOP:
CMP    R2, R3
BLT    DIV_END
SUB    R2, R2, R3
ADD    R1, R1, #1
B      DIV_LOOP
DIV_END:
ADD    R5, R5, R2
MOV    R0, R1
B      MAIN_LOOP
STOP:
B      STOP

```

Registers
 Refresh

r0	00000000
r1	00000000
r2	00000001
r3	0000000a
r4	00000000
r5	0000000a
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	0000003c
cpsr	600001d3

 NZCVI SVC

Disassembly (Ctrl-D)
 Go to address, label, or register: 00000000

Address	Opcode	Disassembly
00000010	e3a01000	11 mov r1, #0 ; 0x0
00000014	e1a02000	12 mov r2, r0
00000018	e3a0300a	13 mov r3, #10 ; 0xa
15 DIV_LOOP:		
DIV_LOOP:		
0000001c	e1520003	16 cmp r2, r3
17 BLT DIV_END		
00000020	ba000002	18 blt 0x30 (0x30: DIV_END)
00000024	e0422003	18 sub r2, r2, r3
00000028	e2811001	19 add r1, r1, #1 ; 0x1
20 B DIV_LOOP		
0000002c	ea000000	20 b 0x1c (0x1c: DIV_LOOP)
22 DIV_END:		
DIV_END:		
00000030	e0855002	23 add r5, r5, r2
00000034	e1a00001	24 mov r0, r1
25 B MAIN_LOOP		
00000038	ea000000	25 b 0x8 (0x8: MAIN_LOOP)
27 STOP:		
28 B STOP		
0000003c	ea000000	28 b 0x3c (0x3c: STOP)
_end:		
00000040	aaaaaa00	bge 0xfeaaaaaf0
00000044	aaaaaa00	bge 0xfeaaaaaf4
00000048	aaaaaa00	bge 0xfeaaaaaf8
0000004c	aaaaaa00	bge 0xfeaaaaafc

Editor (Ctrl-E) Disassembly (Ctrl-D) Memory (Ctrl-M)

Messages
 Compiling...
 Code and data loaded from ELF executable into memory. Total size is 64 bytes.
 Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfpu=neon-fp16 --gdwarf2
 Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmWFZITC.s.elf work/asmWFZIT
 Compile succeeded.

5 **ASSIGNMENT QUESTIONS:**

i) Write a program to swap the first and last character of a given string.

Program and Results Screenshot:

.global _start

_start:

LDR R0, =STRING

MOV R1, R0

FIND_END:

LDRB R2, [R1], #1

CMP R2, #0

BNE FIND_END

SUB R1, R1, #2

LDRB R3, [R0]

LDRB R4, [R1]

STRB R4, [R0]

STRB R3, [R1]

STOP:

B STOP

STRING:

.asciz "HELLO"

Refresh

r0	0000002c
r1	00000030
r2	00000000
r3	00000048
r4	0000004f
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	00000028
cpsr	600001d3

Registers

Call stack

Trace

Breakpoints

Watchpoints

Symbols

Counters

Settings

Number Display Options

Size: Word

Format: Hexadecimal

Memory words per row: 4

Disassembly (Ctrl-D)

Go to address, label, or register: 00000000

Address	Opcode	Disassembly
fffffffc	aaaaaaaa	bge 0xfeaaaaac
		2 .global _start
		4 _start:
		5 LDR R0, =STRING
		_start:
00000000	e59f002c	ldr r0, [pc, #44] ; 0x34
00000004	e1a01000	6 mov r1, r0
		8 FIND_END:
		FIND_END:
00000008	e4d12001	9 ldrb r2, [r1], #1
0000000c	e3520000	10 cmp r2, #0 ; 0x0
		11 BNE FIND_END
00000010	1affffffc	bne 0x8 (0x8: FIND_END)
00000014	e2411002	13 sub r1, r1, #2 ; 0x2
00000018	e5d03000	15 ldrb r3, [r0]
0000001c	e5d14000	16 ldrb r4, [r1]
00000020	e5c04000	18 strb r4, [r0]
00000024	e5c13000	19 strb r3, [r1]
		21 STOP:
		22 B STOP
		STOP:
00000028	eaffffffe	b 0x28 (0x28: STOP)
		STRING:
0000002c	4c4c4548	mcrmi p5, 4, r4, r12, cr8
00000030	0000004f	andeq r0, r0, pc, ASR #32
		5 LDR R0, =STRING
		andeq r0, r0, r12, LSR #32

Editor (Ctrl-E)

Disassembly (Ctrl-D)

Memory (Ctrl-M)

Messages

Compiling...

Code and data loaded from ELF executable into memory. Total size is 56 bytes.

Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfp=neon-fp16 --gdwarf2

Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmUYKL9o.s.elf work/asmUYKL9

Compile succeeded.

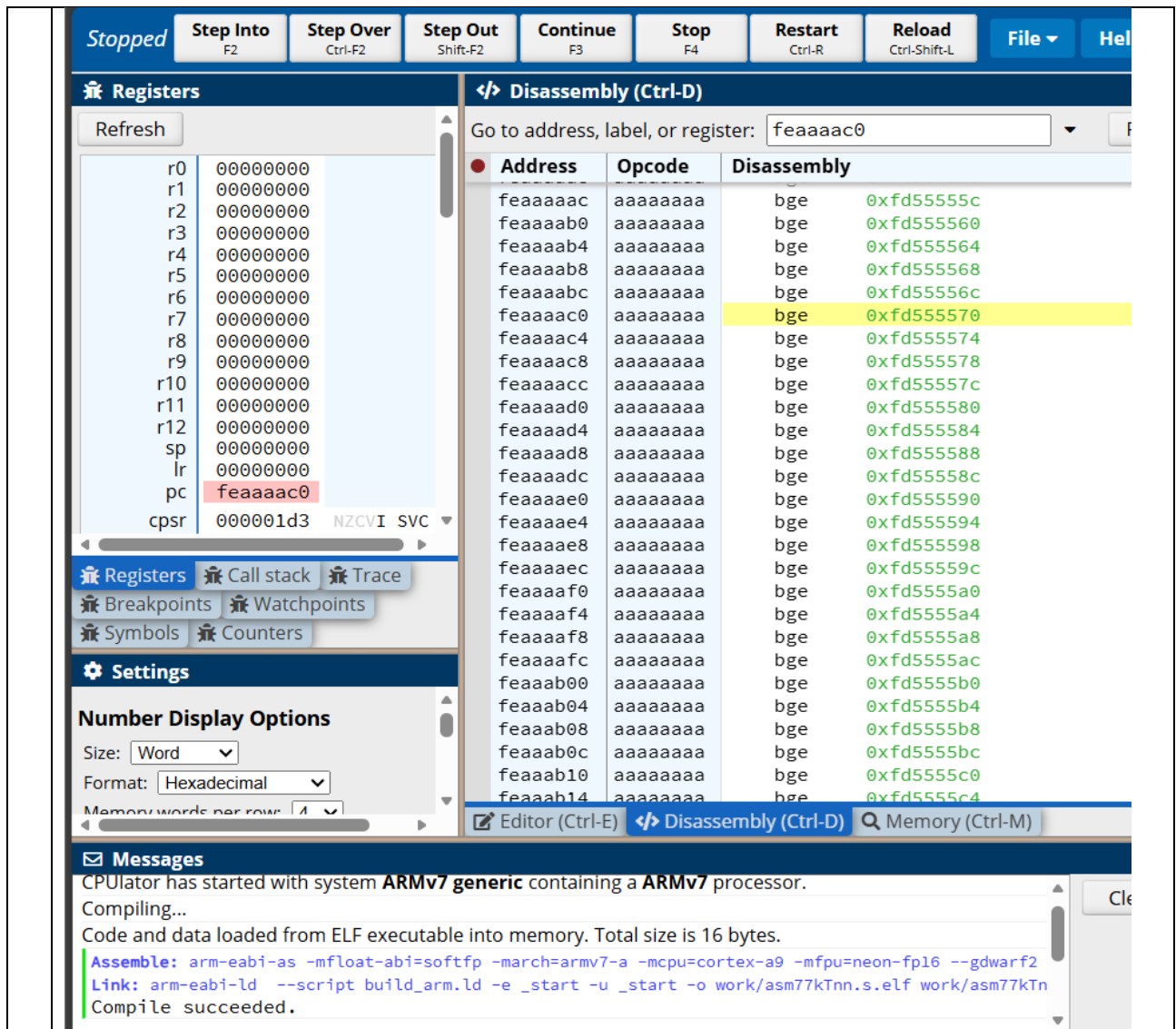
ii) Given a c Code convert it in its equivalent Arm Code : $z = (a \ll 2) \mid (b \& 15)$;

Program and Results Screenshot:

```

LSL    R3, R1, #2
AND    R4, R2, #15
ORR    R0, R3, R4

```

Registers

r0	00000000
r1	00000000
r2	00000000
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	feaaaaac0
cpsr	000001d3 NZCVI SVC

Disassembly (Ctrl-D)

Go to address, label, or register: feaaaaac0

Address	Opcode	Disassembly
feaaaaac	aaaaaaaa	bge 0xfd55555c
feaaaab0	aaaaaaaa	bge 0xfd555560
feaaaab4	aaaaaaaa	bge 0xfd555564
feaaaab8	aaaaaaaa	bge 0xfd555568
feaaaabc	aaaaaaaa	bge 0xfd55556c
feaaaaac0	aaaaaaaa	bge 0xfd555570
feaaaaac4	aaaaaaaa	bge 0xfd555574
feaaaaac8	aaaaaaaa	bge 0xfd555578
feaaaacc	aaaaaaaa	bge 0xfd55557c
feaaaad0	aaaaaaaa	bge 0xfd555580
feaaaad4	aaaaaaaa	bge 0xfd555584
feaaaad8	aaaaaaaa	bge 0xfd555588
feaaaadc	aaaaaaaa	bge 0xfd55558c
feaaaae0	aaaaaaaa	bge 0xfd555590
feaaaae4	aaaaaaaa	bge 0xfd555594
feaaaae8	aaaaaaaa	bge 0xfd555598
feaaaaec	aaaaaaaa	bge 0xfd55559c
feaaaaf0	aaaaaaaa	bge 0xfd5555a0
feaaaaf4	aaaaaaaa	bge 0xfd5555a4
feaaaaf8	aaaaaaaa	bge 0xfd5555a8
feaaaafc	aaaaaaaa	bge 0xfd5555ac
feaaab00	aaaaaaaa	bge 0xfd5555b0
feaaab04	aaaaaaaa	bge 0xfd5555b4
feaaab08	aaaaaaaa	bge 0xfd5555b8
feaaab0c	aaaaaaaa	bge 0xfd5555bc
feaaab10	aaaaaaaa	bge 0xfd5555c0
feaaab14	aaaaaaaa	bge 0xfd5555c4

Messages

CPUlator has started with system **ARMv7 generic** containing a **ARMv7** processor.

Compiling...

Code and data loaded from ELF executable into memory. Total size is 16 bytes.

Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfpu=neon-fp16 --gdwarf2

Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asm77kTnn.s.elf work/asm77kTn

Compile succeeded.

- 6 Write an ARM7TDMI assembly language program in which the main program calls a procedure **STRING_PROC**. The procedure **STRING_PROC** calls separate procedures **LENGTH** to find the length of a string and **REVERSE** to reverse the string. The reversed string must be stored in a separate memory location.

Program and Results Screenshot:

```
.global _start
_start:
    LDR    R0, =STRING
    LDR    R1, =REV_STR
    BL     STRING_PROC
STOP:
    B      STOP
```

STRING_PROC:

```
STMFD SP!, {R4, R5, LR}
MOV R4, R0
BL LENGTH
MOV R5, R0
MOV R0, R4
MOV R2, R5
BL REVERSE
LDMFD SP!, {R4, R5, PC}
```

LENGTH:

```
STMFD SP!, {R1, LR}
MOV R1, #0
```

LEN_LOOP:

```
LDRB R2, [R0, R1]
CMP R2, #0
BEQ LEN_DONE
ADD R1, R1, #1
B LEN_LOOP
```

LEN_DONE:

```
MOV R0, R1
LDMFD SP!, {R1, PC}
```

REVERSE:

```
STMFD SP!, {R3, R4, R5, LR}
SUB R2, R2, #1
MOV R3, #0
```

REV_LOOP:

```
CMP R3, R2
BGT REV_DONE
LDRB R4, [R0, R2]
STRB R4, [R1, R3]
ADD R3, R3, #1
SUB R2, R2, #1
B REV_LOOP
```

REV_DONE:

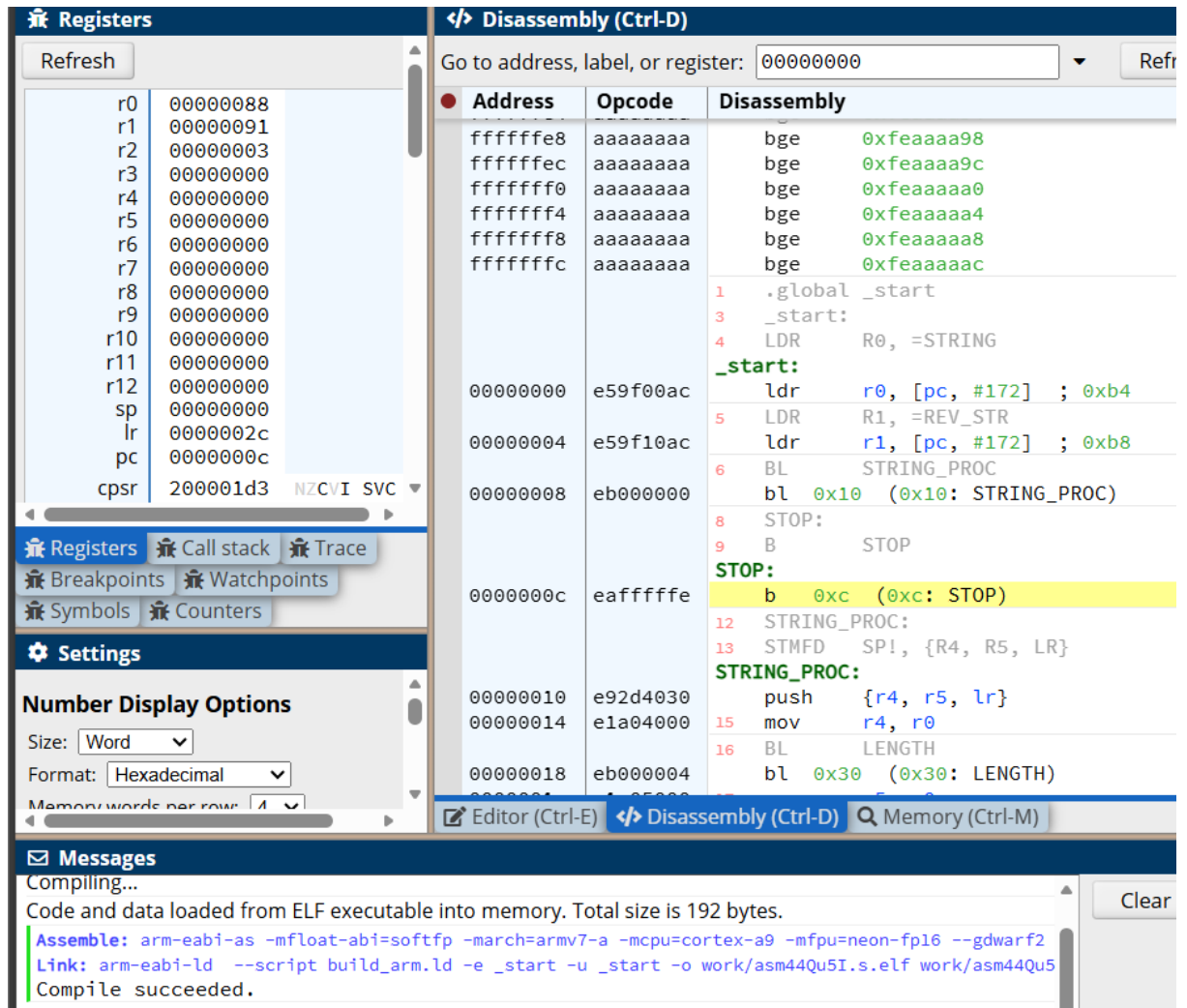
```
MOV R4, #0
STRB R4, [R1, R3]
LDMFD SP!, {R3, R4, R5, PC}
```

STRING:

.asciz "ARM7TDMI"

REV_STR:

.space 32



The screenshot displays a debugger interface with the following components:

- Registers Panel:** Lists registers r0 through pc. The pc register is highlighted with the value 0000000c. Below the list are tabs for Registers, Call stack, Trace, Breakpoints, Watchpoints, Symbols, and Counters.
- Disassembly Panel:** Shows assembly code starting at address 00000000. The code includes instructions like LDR, BL, and STOP. The STOP instruction at address 0000000c is highlighted in yellow.
- Messages Panel:** Displays compilation and linking messages. The messages indicate that the code and data were loaded from an ELF executable into memory (total size 192 bytes) and that the compilation succeeded.

Disassembly Details:

Address	Opcode	Disassembly
ffffffe8	aaaaaaaa	bge 0xfeaaaa98
ffffffec	aaaaaaaa	bge 0xfeaaaa9c
fffffff0	aaaaaaaa	bge 0xfeaaaaa0
fffffff4	aaaaaaaa	bge 0xfeaaaaa4
fffffff8	aaaaaaaa	bge 0xfeaaaaa8
fffffffc	aaaaaaaa	bge 0xfeaaaaac
00000000	e59f00ac	ldr r0, [pc, #172] ; 0xb4
00000004	e59f10ac	ldr r1, [pc, #172] ; 0xb8
00000008	eb000000	bl 0x10 (0x10: STRING_PROC)
0000000c	eaffffff	STOP: b 0xc (0xc: STOP)
00000010	e92d4030	push {r4, r5, lr}
00000014	e1a04000	mov r4, r0
00000018	eb000004	bl 0x30 (0x30: LENGTH)

Messages:

```

Compiling...
Code and data loaded from ELF executable into memory. Total size is 192 bytes.
Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfp=neon-fp16 --gdwarf2
Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asm44Qu5I.s.elf work/asm44Qu5
Compile succeeded.
  
```